Probabilistic World Models for Autonomous Driving

Graduate Project Presentation

1. Introduction: The World Model Concept

This project implements a **World Model** inspired by "World Models" (Ha & Schmidhuber, 2018) and "Dreamer" (Hafner et al., 2019). The core idea is to learn a compressed, predictive model of the environment that allows an agent (or vehicle) to "dream" potential futures and plan accordingly.

The system consists of two primary components:

1. **Vision Model (V)**: Compresses high-dimensional sensory data (images) into a low-dimensional latent state (z_t).
2. **Memory Model (M)**: Predicts the future latent state (z_{t+1}) and vehicle dynamics based on the past.

---

2. Data: The KITTI Odometry Benchmark

2.1 Dataset Overview

We utilize the **KITTI Odometry Benchmark**, a standard dataset for autonomous driving research. It consists of 22 sequences of real-world driving data, containing:

- **Stereo Images**: Left and right camera views (we use the left camera).
- **Ground Truth Poses**: High-precision GPS/IMU localization data.

2.2 Learning the Derivative (Pose Deltas)

A critical design choice is how we represent the vehicle's movement. The raw data provides **Absolute Poses** (Global X, Y, Z coordinates). However, training a model to predict absolute coordinates is prone to overfitting (it memorizes the map).

Instead, we train the model on the **Derivative** of the position, or **Pose Deltas**:

$$\Delta P_t = P_{t+1} - P_t$$

(Technically, we compute the relative transformation matrix between frames).

Why?

- **Generalization**: "Moving forward at 10 m/s" looks the same on Highway A as it does on Highway B. "Being at coordinate (500, 200)" is specific to Highway A.

- **Stationarity**: The distribution of velocity/acceleration is stable, whereas absolute position is unbounded.

---

3. Visual Compression: Variational Autoencoder (VAE)

3.1 Mathematical Formulation

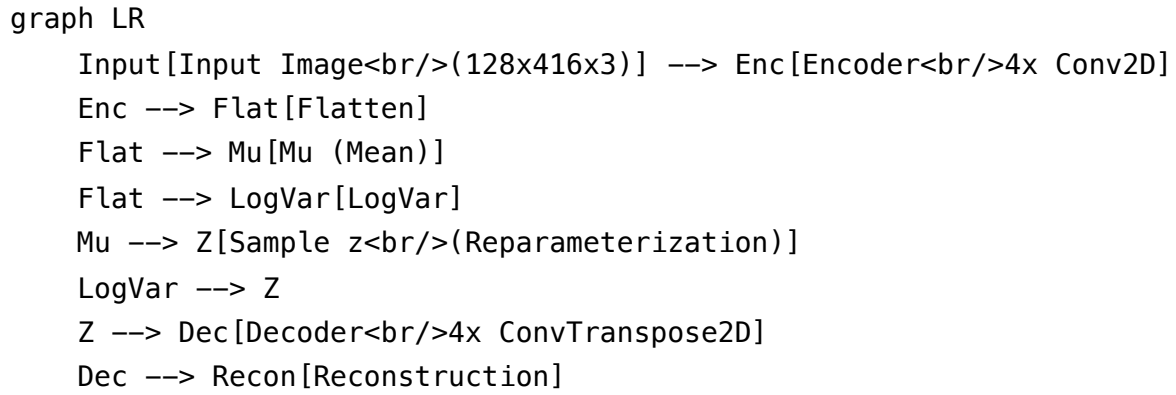
The VAE learns a probabilistic mapping from the image space x to a latent space z. We maximize the Evidence Lower Bound (ELBO):

$$\mathcal{L}_{VAE} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - \beta D_{KL}(q(z|x)||p(z))$$

- **Reconstruction Term** $\mathbb{E}[\log p(x|z)]$: Enforces that the latent code z captures enough information to reconstruct the image. We assume a Gaussian likelihood, which leads to the Mean Squared Error (MSE) loss.
- **Regularization Term** D_{KL}: Forces the learned distribution $q(z|x)$ to approximate a standard Normal prior $p(z) = \mathcal{N}(0, I)$. This ensures a smooth latent space suitable for sampling.

3.2 Architecture

We use a **Convolutional VAE** with 4 downsampling layers.



3.3 Implementation Details

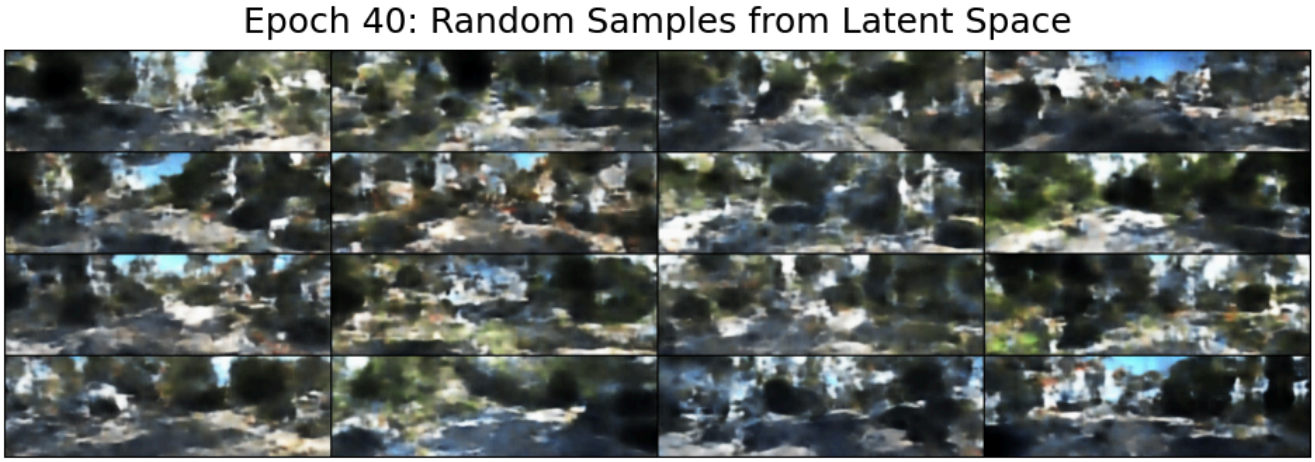
The `ConvVAE` class in `src/models/conv_vea.py` implements the reparameterization trick to allow backpropagation through the stochastic sampling step:

```python
def reparameterize(self, mu, logvar):
    std = torch.exp(0.5 * logvar)
    eps = torch.randn_like(std)
    return mu + eps * std  # z = mu + sigma * epsilon
```

3.4 Results

Reconstruction Quality (Epoch 40):

Latent Space Sampling:



---

4. Latent Dynamics: MDN-RNN

4.1 Mathematical Formulation

The Memory Model predicts the probability distribution of the next latent state z_{t+1} given the current state z_t and hidden state h_t. Since the future is uncertain and multi-modal, we model it as a **Mixture of Gaussians (GMM)** using a Mixture Density Network (MDN).

$$P(z_{t+1}|h_t) = \sum_{k=1}^{K} \pi_k(h_t) \mathcal{N}(z_{t+1}|\mu_k(h_t), \sigma_k(h_t))$$

- π_k: Mixing coefficients (probabilities of each Gaussian).
- μ_k, σ_k: Mean and variance of each Gaussian component.

The loss function is the **Negative Log Likelihood (NLL)** of the true next state under this predicted distribution:

$$\mathcal{L}_{MDN} = -\log \left(\sum_{k=1}^{K} \pi_k \exp \left(-\frac{(z_{t+1} - \mu_k)^2}{2\sigma_k^2} \right) \right)$$

4.2 Architecture Explained

The **DreamerMDRNN** is composed of three distinct parts working in unison:

1. **The Memory (LSTM)**:
    - Acts as the brain of the model. It receives the compressed visual information (z_t) and updates its internal hidden state (h_t).
    - This hidden state represents the "context" of the drive—it remembers velocity, acceleration, and past events.
2. **The Vision Heads (MDN)**:
    - Because the future is uncertain (e.g., a car might turn left OR right), we cannot predict a single future frame.
    - Instead, we use a **Mixture Density Network (MDN)** to predict a *probability distribution* of possible futures.
    - It outputs parameters for a Gaussian Mixture Model (GMM): Mixing coefficients (π), Means (μ), and Variances (σ).
3. **The Pose Head**:
    - A simple linear layer that looks at the memory (h_t) and predicts the physical movement of the car ($\Delta x, \Delta y, \Delta z, \Delta roll, \Delta pitch, \Delta yaw$).

```
graph TD
    Zt[Latent z_t] --> LSTM[LSTM Core]
    Ht[Hidden h_t-1] --> LSTM
    LSTM --> Ht_new[Hidden h_t]

    Ht_new --> FC_Pi[FC Pi (Softmax)]
    Ht_new --> FC_Mu[FC Mu]
    Ht_new --> FC_Sigma[FC Sigma (Exp)]

    FC_Pi --> GMM[GMM Distribution<br/>P(z_t+1 | z_t)]
    FC_Mu --> GMM
    FC_Sigma --> GMM

    Ht_new --> FC_Pose[FC Pose]
    FC_Pose --> Pose[Delta Pose<br/>(dx, dy, ...)]
```

4.3 The Training Process

We train the model using a **Stateless** approach to ensure stability and efficiency.

1. **Input Sequence**: We feed the model a short sequence of 5 frames (e.g., $t = 0$ to $t = 4$).
2. **Forward Pass**:
    - The LSTM processes these 5 frames sequentially.
    - At each step, it predicts the *next* latent state (z_{t+1}) and the *next* movement.
3. **Loss Calculation**:
    - **Vision Loss**: We check if the *actual* next frame (z_{t+1}) falls within the predicted probability distribution (high likelihood = good).
    - **Pose Loss**: We measure the Mean Squared Error (MSE) between the predicted movement and the actual movement.
4. **Backpropagation**:
    - The error signal travels backwards through time (BPTT).
    - It teaches the LSTM to update its memory gates so that it pays attention to relevant features (like velocity changes) in the future.

4.4 Implementation Details

The loss function handles numerical stability using the Log-Sum-Exp trick (`torch.logsumexp`).

```python
# src/models/mdnrnn_pose.py

def loss_function(self, y_true_latent, ...):
    # ... (Calculation of log_prob for each Gaussian) ...

    # Log-Sum-Exp for numerical stability
    # log(sum(exp(x))) = logsumexp(x)
    weighted_log_prob = log_pi + log_prob
    log_prob_total = torch.logsumexp(weighted_log_prob, dim=2)

    loss_mdn = -torch.mean(log_prob_total)
    return loss_mdn + (loss_pose * pose_weight)
```

4.5 Training Strategy & Limitations

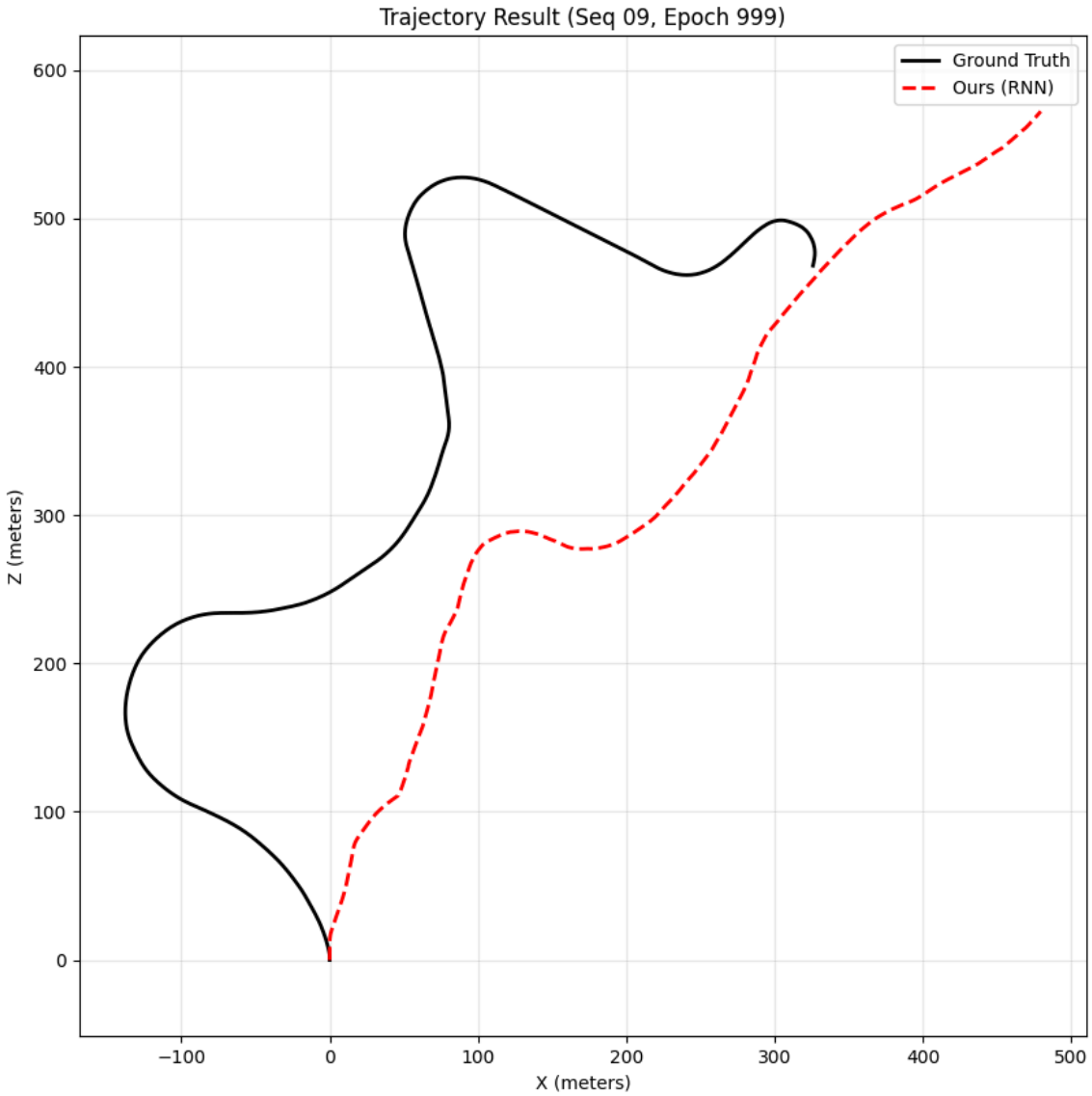
Stateless Training with Windowing: We train the model on short, shuffled sequences (Window Size = 5) where the hidden state is reset at the start of each batch.

- **Advantage**: Stabilizes gradients and allows for random sampling of the dataset (I.I.D. assumption).
- **Limitation (Training-Inference Mismatch)**: The model is trained to have a "short-term memory" (5 steps) but is evaluated on long sequences (1000+ steps). This can lead to **drift** over time, as the model may not learn to manage long-term memory slots effectively.
- **Future Work**: Implement **Truncated Backpropagation Through Time (TBPTT)** with state passing to allow the model to learn global map consistency.

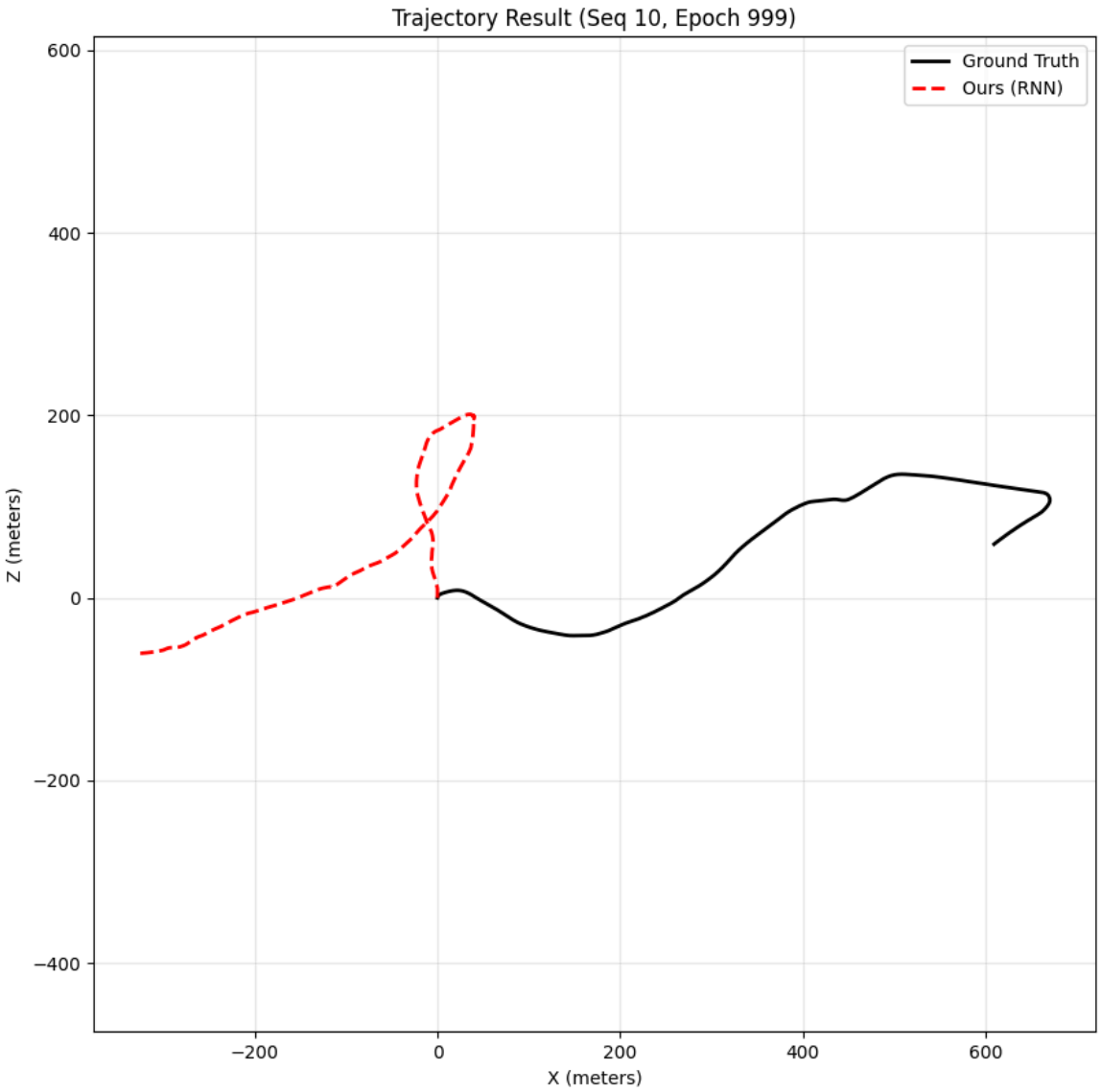
4.6 Results: Trajectory Prediction

By recursively feeding the predicted z_{t+1} back into the RNN ("dreaming") and integrating the predicted pose deltas, we can reconstruct the vehicle's trajectory.

Test Sequence 09 (Ground Truth vs. Prediction):



Test Sequence 10:



---

5. Conclusion

This project demonstrates a functional World Model capable of:

1. Compressing visual information into a meaningful latent space.
2. Learning the probabilistic dynamics of the environment.
3. Accurately predicting vehicle odometry and future states.

This architecture forms the foundation for Model-Based Reinforcement Learning, where an agent can learn policies entirely within this "dream" environment.